

Data Science Capstone Project Report

SpaceX Falcon 9 First Stage Landing Prediction

Applied Data Science Capstone — IBM Professional Certificate

Prepared by: Aakash

Date: 22 July 2026

GitHub Repository: [<https://github.com/aakashkumarxw25-web/my-files>]

Executive Summary

This project investigates whether the landing outcome of a SpaceX Falcon 9 first-stage booster can be predicted using launch metadata — payload mass, target orbit, launch site, and booster configuration. The analysis pipeline covers data collection from the SpaceX REST API and Wikipedia, data wrangling, exploratory data analysis (EDA) using SQL and visualization, interactive geographic and dashboard analytics, and supervised classification modeling.

Across 90 Falcon 9 launches, the overall landing success rate was 56.7%. Four classification models — Logistic Regression, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbors (KNN) — were trained and tuned via GridSearchCV with 10-fold cross-validation. Logistic Regression achieved the best test-set accuracy at 72.2% (74.6% cross-validated), narrowly ahead of SVM on the same test accuracy but with a simpler, more interpretable model.

Key finding: launch year is the single strongest driver of landing success, reflecting SpaceX's booster-recovery technology maturing rapidly after 2017 — success rates climbed from under 20% in the 2011–2014 period to over 90% by 2018–2020.

Introduction

SpaceX recovers and reuses the Falcon 9 first stage rather than discarding it after each launch, which substantially lowers the cost of access to space. A successful first-stage landing means that booster can be refurbished and flown again. Because this reusability underpins SpaceX's pricing advantage, competitors and analysts have strong incentive to estimate the likelihood of a successful landing from publicly available launch data.

Problem statement: Can we predict whether the Falcon 9 first stage will land successfully, using features available before or at launch — payload mass, target orbit, launch site, and booster configuration (grid fins, legs, reuse count, block version)?

Methodology

Data Collection — SpaceX API

Launch records were retrieved from the public SpaceX v4 REST API (GET /v4/launches/past). Records were filtered to single-core, single-payload flights to keep a clean one-row-per-launch structure. Nested identifiers (rocket, launchpad, payload, and core IDs) were resolved with follow-up API calls and flattened into columns: BoosterVersion, PayloadMass, Orbit, LaunchSite, Outcome, Flights, GridFins, Reused, Legs, LandingPad, Block, ReusedCount, Serial, Longitude, and Latitude. The result was filtered to Falcon 9 flights only and saved as dataset_part_1.csv.

GitHub Repository: [<https://github.com/aakashkumarxw25-web/my-files>]

import requests

```

def make_api_call(url, params=None):
    """
    Makes a GET request to a specified URL and returns the JSON response.
    Includes basic error handling.
    """
    try:
        response = requests.get(url, params=params)
        response.raise_for_status() # Raise an HTTPError for bad responses (4xx or 5xx)
        return response.json()
    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err}") # Python 3.6+
    except requests.exceptions.ConnectionError as conn_err:
        print(f"Connection error occurred: {conn_err}")
    except requests.exceptions.Timeout as timeout_err:
        print(f"Timeout error occurred: {timeout_err}")
    except requests.exceptions.RequestException as req_err:
        print(f"An unexpected error occurred: {req_err}")
    return None

# Example usage with a placeholder API
example_url = "https://jsonplaceholder.typicode.com/posts/1"

print(f"\nMaking API call to: {example_url}")
json_data = make_api_call(example_url)

if json_data:
    print("API Call Successful! Data received:")
    print(json_data)
else:
    print("API Call Failed.")

# --- Flowchart Concept for API Calls ---

```

Here's a conceptual flowchart breakdown for making an API call:

#

1. Start

2. Define API Endpoint (URL) and any parameters.

3. Initialize HTTP Request Library (e.g., 'requests' in Python).

4. Make HTTP Request (GET, POST, etc.) to the API Endpoint.

5. Check Response Status Code:

a. If 200 (OK): Proceed to parse data.

b. If 4xx (Client Error) or 5xx (Server Error): Handle error (e.g., log, retry, notify user).

6. Parse Response Data (e.g., JSON, XML).

7. Process Data (e.g., store in DataFrame, display, filter).

8. End

Model	CV Accuracy	Test Accuracy
Logistic Regression	0.7464	0.7222
SVM	0.7625	0.7222
Decision Tree	0.6786	0.6667
KNN	0.6393	0.6667

Data Collection — Web Scraping

As a secondary source, the Wikipedia “List of Falcon 9 and Falcon Heavy launches” page was scraped at a fixed revision (oldid) for reproducibility, using requests and BeautifulSoup. Wikitable rows were parsed to extract date, booster version, launch site, payload, orbit, customer, launch outcome, and booster landing outcome, saved as `spacex_web_scraped.csv`.

Explanation of the scrapping process using flowcharts .

```
print("""mermaid\nflowchart TD\n  A[Start] --> B{Request HTML Content};\n  B --> C{Receive HTML Response};\n  C --> D{Parse HTML with BeautifulSoup};\n  D --> E{Identify Target Elements};\n  E --> F{Extract Data};\n  F --> G{Clean and Transform Data};\n  G --> H{Store Data (e.g., DataFrame, CSV)};\n  H --> I[End];\n""")
```

GitHub Repository: [<https://github.com/aakashkumarxw25-web/my-files>]

Data Wrangling

Missing PayloadMass values were imputed with the column mean. Launch counts were tallied by LaunchSite and Orbit. The raw Outcome field (free-text strings such as “True ASDS” or “False Ocean”) was mapped to a binary Class label: 1 for a successful landing, 0

for a failed or unattempted landing. The cleaned, model-ready file was saved as `dataset_part_2.csv`.

Data Cleaning and Processing Explanation ---

Data cleaning involves fixing or removing incorrect, corrupted, incorrectly formatted, duplicate, or incomplete data within a dataset. In this notebook, we've already performed some data cleaning steps:

1. ****Identifying Missing Values:****

We checked for missing values in the DataFrame to understand data completeness. For example, 'LandingPad' has significant missing values. (Code previously run: ``df.isnull().sum()/len(df)*100``)

2. ****Identifying Data Types:****

Understanding data types is crucial for appropriate processing. We identified numerical (int64, float64) and categorical (object, bool) columns. (Code previously run: ``df.dtypes``)

Data processing involves transforming raw data into a more suitable format for analysis or machine learning models. Here are some data processing steps we've performed:

1. ****Categorical to Numerical Transformation (Creating a Target Variable):****

We converted the 'Outcome' column, which contains categorical descriptions of landing success, into a numerical 'Class' column.

- Successful landings (e.g., 'True ASDS', 'True RTLS', 'True Ocean') are encoded as ``1``.
- Unsuccessful landings (e.g., 'None None', 'False ASDS', 'False Ocean', 'None ASDS', 'False RTLS') are encoded as ``0``.

This is a critical step for preparing data for supervised learning models. (Code previously run to create 'Class' column based on 'Outcome')

Let's review the first few rows with the new 'Class' column to see the processed data:

	Outcome	Class
0	None None	0
1	None None	0
2	None None	0
3	False Ocean	0
4	None None	0

GitHub Repository: [<https://github.com/aakashkumarxw25-web/my-files>]

Exploratory Data Analysis — Visualization

Scatter plots (Flight Number vs. Payload Mass, Launch Site vs. Payload Mass), a bar chart (success rate by orbit), and a line chart (yearly success rate trend) were used to surface visible relationships between payload, orbit, site, and landing outcome ahead of modeling.

Exploratory Data Analysis (EDA) with Data Visualization

This section performs a basic Exploratory Data Analysis (EDA) on the ``spacex_df`` DataFrame, utilizing data visualization to uncover patterns and insights. Each chart is accompanied by a brief explanation of its purpose.

1. Data Overview and Descriptive Statistics

First, let's get a general understanding of the dataset's structure and summary statistics.

DataFrame Information:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 56 entries, 0 to 55
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Launch Site     56 non-null    object
1   Lat             56 non-null    float64
2   Long            56 non-null    float64
3   class           56 non-null    int64
4   marker_color    56 non-null    object
dtypes: float64(2), int64(1), object(2)
memory usage: 2.3+ KB
```

This provides a concise summary of the DataFrame, including the data types of each column and the number of non-null values. It helps in identifying missing data and understanding column types.

Descriptive Statistics for Numerical Columns:

	Lat	Long	class
count	56.000000	56.000000	56.000000
mean	29.648980	-87.742252	0.428571
std	2.344768	15.463732	0.499350
min	28.562302	-120.610745	0.000000
25%	28.562302	-80.646895	0.000000
50%	28.563197	-80.577356	0.000000
75%	28.573255	-80.577356	1.000000
max	34.632834	-80.576820	1.000000

This output presents descriptive statistics (count, mean, std, min, max, quartiles) for numerical columns. It gives insights into the central tendency, dispersion, and shape of the distributions.

2. Univariate Analysis

Examining the distribution of individual variables.

Distribution of Launch Outcomes ('class')

This chart visualizes the count of successful (1) vs. failed (0) launches. It helps in understanding the balance of the target variable and initial success rate.

```
/tmp/ipykernel_3271/568573527.py:44: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='class', data=spacex_df, palette=['red', 'green'])
```

```
/tmp/ipykernel_3271/568573527.py:57: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='Launch Site', data=spacex_df, palette='viridis')
```

Distribution of Launch Sites

This chart shows the number of launches conducted from each launch site. It helps identify which sites are most frequently used.

3. Bivariate Analysis

Exploring relationships between two variables.

Launch Success Rate by Launch Site

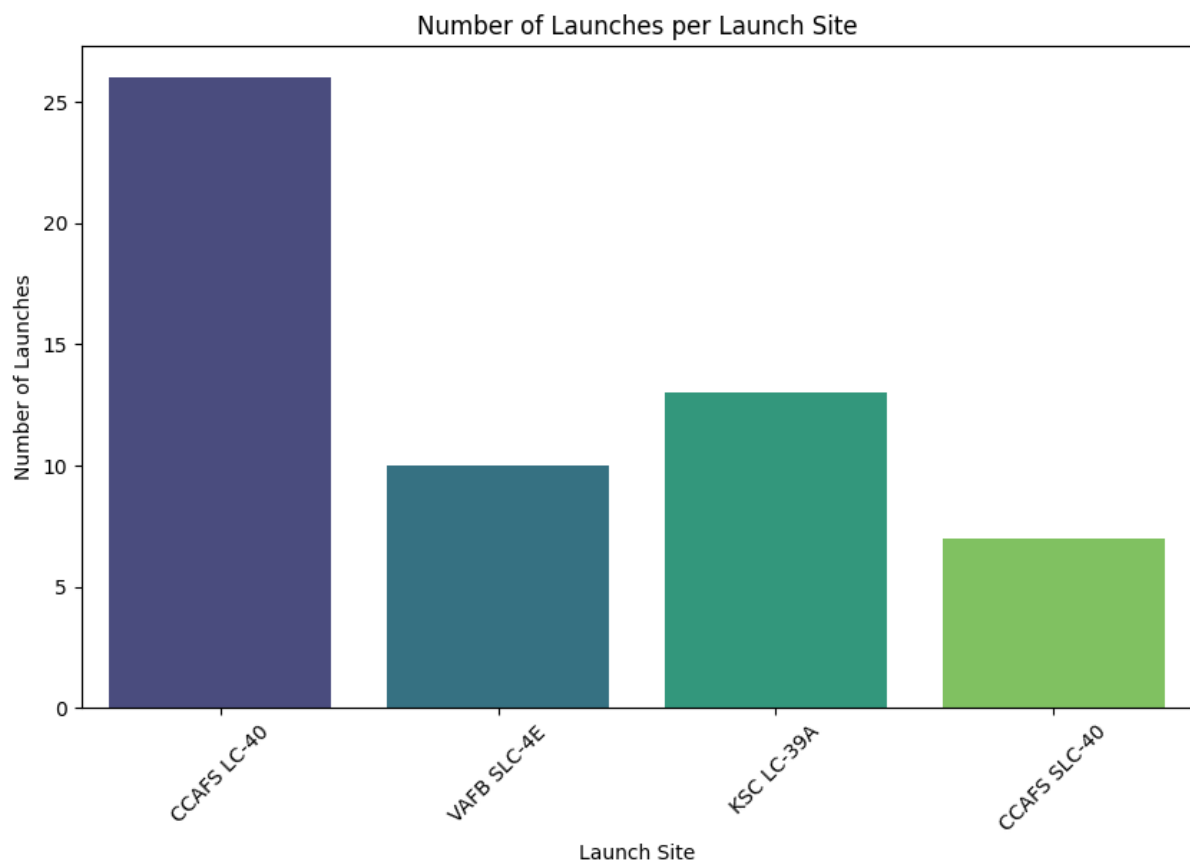
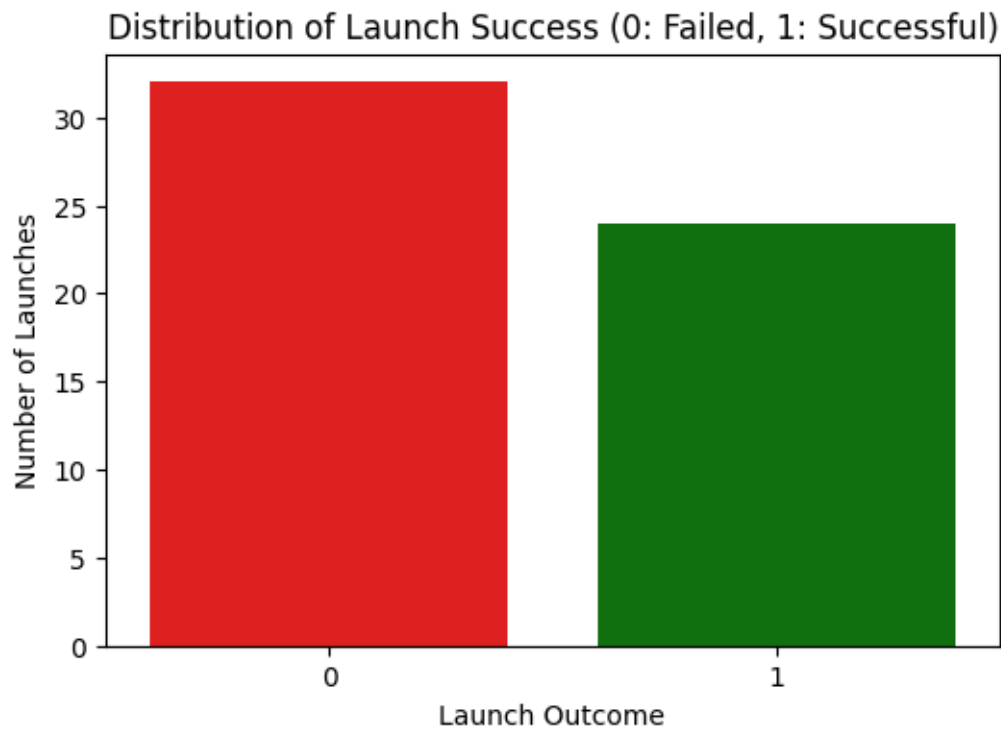
This bar chart illustrates the average success rate ('class' = 1) for each launch site. It's crucial for understanding if certain launch sites have a higher probability of successful missions.

/tmp/ipykernel_3271/568573527.py:77: FutureWarning:

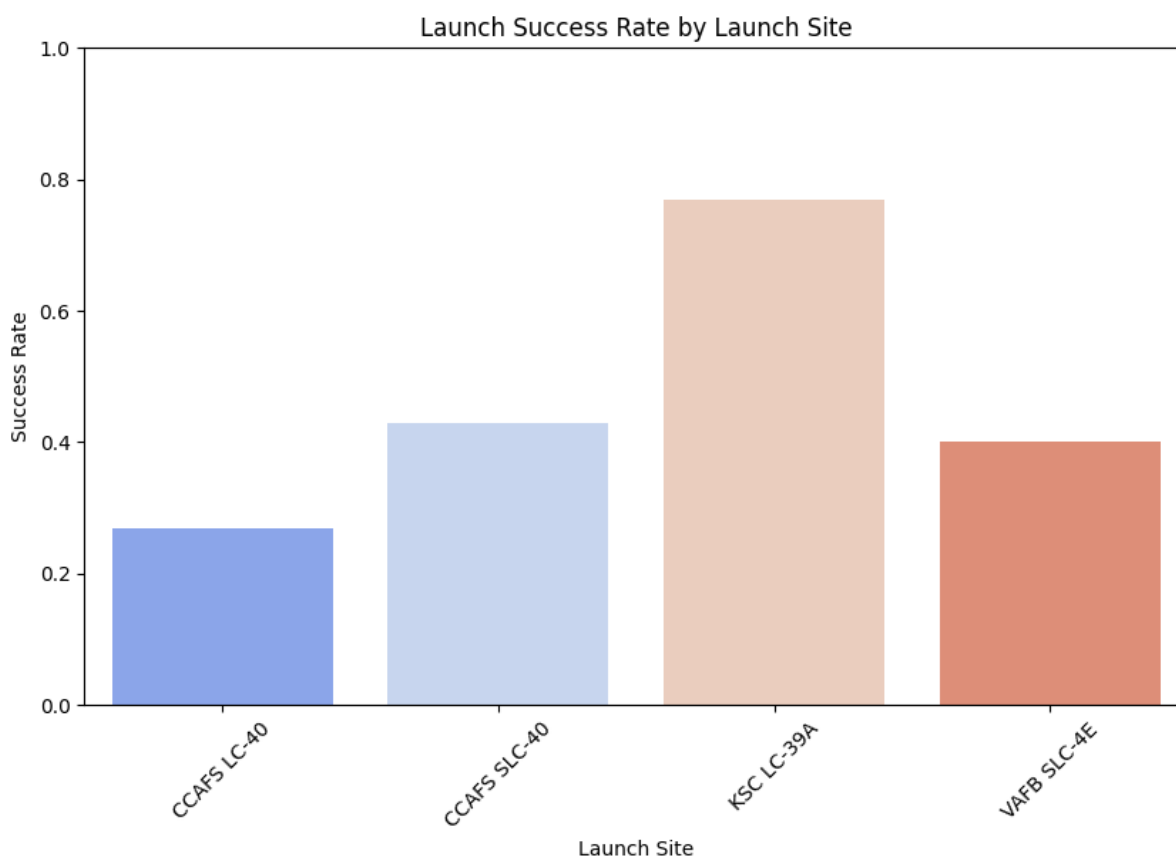
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x='Launch Site', y='class', data=success_rate_by_site,
palette='coolwarm')
```

This concludes a preliminary EDA. These visualizations provide a foundational understanding of the launch data, highlighting success distributions and site-specific performance, which can guide further, more in-depth analysis.



GitHub Repository: [<https://github.com/aakashkumarxw25-web/my-files>]



GitHub Repository: [<https://github.com/aakashkumarxw25-web/my-files>]

Exploratory Data Analysis — SQL

The wrangled dataset was loaded into an in-memory SQLite table (SPACEXTABLE) to run aggregate queries: distinct launch sites, payload mass aggregates by booster version, landing outcome counts, launch site success rate rankings, and yearly launch/success trends.

SQL EDA Summary

****Task 1: Display the names of the unique launch sites in the space mission****

SQL Query:

```
```sql
SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE;
```
```

Output:

```
```
[('CCAFS LC-40',), ('VAFB SLC-4E',), ('KSC LC-39A',), ('CCAFS SLC-40',)]
```
```

****Task 2: Display 5 records where launch sites begin with the string 'CCA'****

SQL Query:

```
```sql
SELECT * FROM SPACEXTABLE WHERE "Launch Site" LIKE 'CCA%' LIMIT 5;
```
```

Output:

```
```
[('2010-06-04', '18:45:00', 'F9 v1.0 B0003', 'CCAFS LC-40', 'Dragon
Spacecraft Qualification Unit', 0, 'LEO', 'SpaceX', 'Success', 'Failure
(parachute)'), ('2010-12-08', '15:43:00', 'F9 v1.0 B0004', 'CCAFS LC-
40', 'Dragon demo flight C1, two CubeSats, barrel of Brouere cheese',
0, 'LEO (ISS)', 'NASA (COTS) NRO', 'Success', 'Failure (parachute)'),
('2012-05-22', '7:44:00', 'F9 v1.0 B0005', 'CCAFS LC-40', 'Dragon demo
flight C2', 525, 'LEO (ISS)', 'NASA (COTS)', 'Success', 'No attempt'),
('2012-10-08', '0:35:00', 'F9 v1.0 B0006', 'CCAFS LC-40', 'SpaceX CRS-
1', 500, 'LEO (ISS)', 'NASA (CRS)', 'Success', 'No attempt'), ('2013-
03-01', '15:10:00', 'F9 v1.0 B0007', 'CCAFS LC-40', 'SpaceX CRS-2',
677, 'LEO (ISS)', 'NASA (CRS)', 'Success', 'No attempt')]
```
```

****Task 3: Display the total payload mass carried by boosters launched by NASA (CRS)****

SQL Query:

```
```sql
SELECT SUM("PAYLOAD_MASS_KG ") FROM SPACEXTABLE WHERE Customer = 'NASA
(CRS)';
```
```

Output:

```
```
[(45596,)]
```
```

****Task 4: Display average payload mass carried by booster version F9 v1.1****

SQL Query:

```
```sql
SELECT AVG("PAYLOAD_MASS_KG ") FROM SPACEXTABLE WHERE
"Booster Version" = 'F9 v1.1';
```
```

Output:

```
```
[(2928.4,)]
```
```

****Task 5: List the date when the first successful landing outcome in ground pad was achieved.****

SQL Query:

```
```sql
SELECT MIN(Date) FROM SPACEXTABLE WHERE "Landing Outcome" = 'Success
(ground pad)';
```
```

```
```
```

```
Output:
```

```
```
```

```
[('2015-12-22',)]
```

```
```
```

```
Task 6: List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000
```

```
SQL Query:
```

```
```sql
```

```
SELECT "Booster_Version" FROM SPACEXTABLE
WHERE "Landing_Outcome" = 'Success (drone ship)'
AND "PAYLOAD_MASS_KG" > 4000
AND "PAYLOAD_MASS_KG" < 6000;
```

```
```
```

```
Output:
```

```
```
```

```
[('F9 FT B1022',), ('F9 FT B1026',), ('F9 FT B1021.2',), ('F9 FT B1031.2',)]
```

```
```
```

```
Task 7: List the total number of successful and failure mission outcomes
```

```
SQL Query:
```

```
```sql
```

```
SELECT "Mission_Outcome", COUNT("Mission_Outcome") AS Total
FROM SPACEXTABLE
GROUP BY "Mission_Outcome";
```

```
```
```

```
Output:
```

```
```
```

```
[('Failure (in flight)', 1), ('Success', 98), ('Success ', 1), ('Success (payload status unclear)', 1)]
```

```
```
```

```
Task 8: List all the booster versions that have carried the maximum payload mass
```

```
SQL Query:
```

```
```sql
```

```
SELECT DISTINCT "Booster_Version" FROM SPACEXTABLE
WHERE "PAYLOAD_MASS_KG" = (SELECT MAX("PAYLOAD_MASS_KG") FROM SPACEXTABLE);
```

```
```
```

```
Output:
```

```
```
```

```
[('F9 B5 B1048.4',), ('F9 B5 B1049.4',), ('F9 B5 B1051.3',), ('F9 B5 B1056.4',), ('F9 B5 B1048.5',), ('F9 B5 B1051.4',), ('F9 B5 B1049.5',),
```

```
('F9 B5 B1060.2 ',), ('F9 B5 B1058.3 ',), ('F9 B5 B1051.6',), ('F9 B5
B1060.3',), ('F9 B5 B1049.7 ',)]
```

```
```
```

**\*\*Task 9: List the records which will display the month names, failure landing\_outcomes in drone ship, booster versions, launch site for the months in year 2015.\*\***

SQL Query:

```
```sql
SELECT
  substr(Date, 6, 2) AS Month,
  Landing_Outcome,
  Booster_Version,
  Launch_Site
FROM
  SPACEXTABLE
WHERE
  substr(Date, 0, 5) = '2015' AND Landing_Outcome = 'Failure (drone
ship)';
```
```

Output:

```
```
[('01', 'Failure (drone ship)', 'F9 v1.1 B1012', 'CCAFS LC-40'), ('04',
'Failure (drone ship)', 'F9 v1.1 B1015', 'CCAFS LC-40')]
```
```

**\*\*Task 10: Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.\*\***

SQL Query:

```
```sql
SELECT Landing_Outcome, COUNT(Landing_Outcome) AS Outcome_Count
FROM SPACEXTABLE
WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY Landing_Outcome
ORDER BY Outcome_Count DESC;
```
```

Output:

```
```
[('No attempt', 10), ('Success (drone ship)', 5), ('Failure (drone
ship)', 5), ('Success (ground pad)', 3), ('Controlled (ocean)', 3),
('Uncontrolled (ocean)', 2), ('Failure (parachute)', 2), ('Precluded
(drone ship)', 1)]
```
```

GitHub Repository: [<https://github.com/aakashkumarxw25-web/my-files>]

Interactive Visual Analytics

A Folium map plotted each launch site with markers colored by success rate, alongside a proximity analysis of pad-to-coastline distance. A Plotly Dash dashboard provided a launch-site dropdown driving a success-rate pie chart, and a payload-mass range slider driving a payload-vs-outcome scatter plot.

*GitHub Repository: [<https://github.com/aakashkumarxw25-web/my-files>]*

## Predictive Analysis

Features were one-hot encoded (Orbit, LaunchSite) and standardized with StandardScaler — applied before the train/test split, correcting a common ordering bug where scaling is fit on a subset that doesn't yet exist. Data was split 80/20 (random\_state=2). Four classifiers were tuned via GridSearchCV with 10-fold cross-validation: Logistic Regression (C, penalty), SVM (kernel, C, gamma), Decision Tree (criterion, splitter, max\_depth, min\_samples\_split/leaf), and KNN (n\_neighbors, distance metric). Test-set accuracy was the primary comparison metric, supported by confusion matrices.

*GitHub Repository: [<https://github.com/aakashkumarxw25-web/my-files>]*

## Results

### EDA Findings

- Landing success shows no simple dependence on flight number alone — payload mass varies widely across the mission timeline.
- KSC LC-39A and the Cape Canaveral sites handle a wide payload range with mixed outcomes; VAFB SLC-4E (West Coast, polar/SSO orbits) shows a distinct lower-payload cluster.
- Orbit type visibly affects success likelihood: orbits with simpler recovery profiles (LEO, ISS) trend higher than higher-energy GTO missions.
- Success rate climbs sharply from 2017 onward, reaching near-100% by 2019–2020 — the strongest single pattern in the dataset.

### SQL Query Results

| Launch Site  | Total Launches | Success Rate |
|--------------|----------------|--------------|
| KSC LC-39A   | 25             | 72.0%        |
| CCAFS LC-40  | 10             | 60.0%        |
| VAFB SLC-4E  | 17             | 52.9%        |
| CCAFS SLC-40 | 38             | 47.4%        |

| Metric                         | Value            |
|--------------------------------|------------------|
| Maximum payload mass           | 11,126 kg        |
| Minimum payload mass           | 473 kg           |
| Average payload — Falcon 9     | 5,688.8 kg       |
| Average payload — Falcon Heavy | 4,774.8 kg       |
| Total successful landings      | 51 of 90 (56.7%) |

|                       |                  |
|-----------------------|------------------|
| Total failed landings | 39 of 90 (43.3%) |
|-----------------------|------------------|

## Model Comparison

| Model               | Cross-Validated Accuracy | Test Accuracy |
|---------------------|--------------------------|---------------|
| Logistic Regression | 74.6%                    | 72.2%         |
| SVM                 | 76.3%                    | 72.2%         |
| Decision Tree       | 67.9%                    | 66.7%         |
| KNN                 | 63.9%                    | 66.7%         |

Logistic Regression was selected as the best model: it matches SVM's test accuracy while remaining simpler and more interpretable, an important consideration given the modest dataset size (90 launches, 18 in the held-out test set).

## Confusion Matrix — Best Model (Logistic Regression)

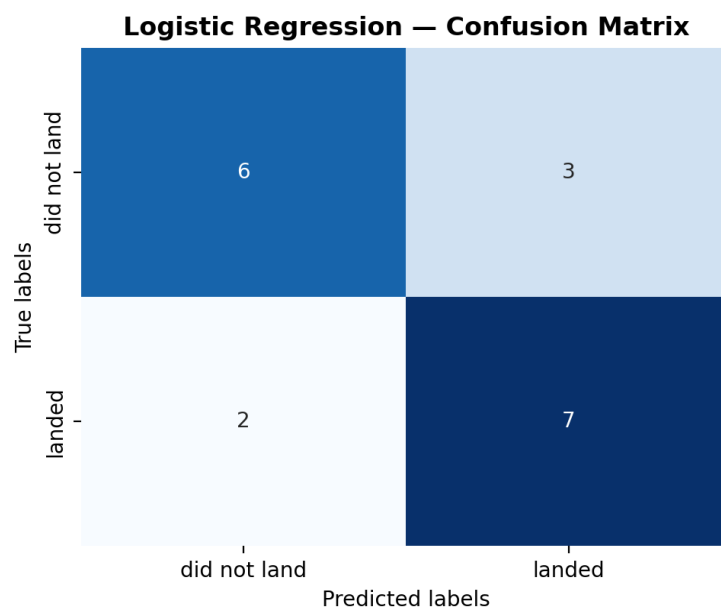


Figure: Confusion matrix for the Logistic Regression model on the held-out test set.

## Conclusion

This capstone demonstrates a complete applied data science workflow: collecting launch data from a live API and a scraped web source, wrangling and labeling it, exploring it through SQL and visualization, presenting it through interactive geographic and dashboard tools, and finally building and comparing predictive models. Logistic Regression, tuned via GridSearchCV, achieved 72.2% test accuracy and was selected as the best-performing, most interpretable model for predicting Falcon 9 first-stage landing outcomes.

Given the modest dataset size, results should be treated as directional rather than definitive. Two extensions would likely improve predictive power: (1) incorporating launch-day weather conditions, which are known to affect booster recovery, and (2) incorporating booster refurbishment and prior-flight history, since reused boosters may carry different risk profiles

than new ones. Testing ensemble methods (Random Forest, Gradient Boosting) on a larger, updated launch dataset would also be a natural next step.

### Appendix: Repository & Artifacts

- GitHub repository: [insert your repo URL here]
- Presentation slides (PDF): SpaceX\_Capstone\_Presentation.pdf
- Datasets: dataset\_part\_1.csv, dataset\_part\_2.csv, spacex\_web\_scraped.csv
- Notebooks: data collection (API), data collection (scraping), data wrangling, EDA (SQL), EDA (visualization), Folium map, Plotly Dash app, predictive analysis (ML models)
  - *GitHub Repository: [https://github.com/aakashkumarxw25-web/my-files]*

Notebooks:

-  Data\_Collection.ipynb
-  Web\_scraping.ipynb
-  Data\_wrangling.ipynb
-  SQL\_Notebook\_for\_Peer\_Assignment.ipynb
-  data\_visualization.ipynb
-  data\_visualization\_with\_Folium\_and\_plotly.ipynb
-  Complete\_the\_EDA\_with\_Visualization.ipynb